Tutorial: Calculadora Otimizada com Expo e TypeScript

Prof. João Siles

Este guia ensina a construir uma calculadora funcional, utilizando React Native, Expo Router e TypeScript.

Pré-Requisitos:

- Node.js instalado.
- Aplicativo Expo Go instalado no celular (opcional)

Inicializando o projeto:

```
npx create-expo-app@latest CalculadoraApp
cd CalculadoraApp
npm run reset-project
```

Codificando a Calculadora:

No arquivo *app/index.tsx* digite o seguinte código:

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, TouchableOpacity, SafeAreaView } from 'react-native';

interface BotaoProps {
  titulo: string;
  corFundo?: string;
  corTexto?: string;
}

export default function Index() {
  const [expressao, setExpressao] = useState<string>('');
  const [resultado, setResultado] = useState<string>('0');

  const operadores = ['+', '-', 'x', '÷', '.'];

  const linhasDeBotoes = [
    ['C', '(', ')', '÷'],
    ['7', '8', '9', 'x'],
    ['4', '5', '6', '-'],
    ['1', '2', '3', '+'],
    ['0', '.', '⌫', '=']
  ];

  const obterCorFundo = (botao: string): string => {
    if (botao === 'C') return '#ff3b30';
    if (botao === '=') return '#34c759';
    if (['÷', 'x', '-', '+'].includes(botao)) return '#ff9500';
    if (['(', ')', '⌫'].includes(botao)) return '#555555';
    return '#333333';
  };

  const lidarComToque = (valor: string): void => {
    if (valor === 'C') {
      setExpressao('');
      setResultado('0');
    } else if (valor === '⌫') {
      const novaExpressao = expressao.slice(0, -1);
      setExpressao(novaExpressao);
      setResultado(novaExpressao.length > 0 ? novaExpressao : '0');
    } else if (valor === '=') {
      try {
        const expressaoFormatada = expressao.replace(/x/g, '*').replace(/÷/g, '/');
        const resultadoCalculado = eval(expressaoFormatada);

        setResultado(String(resultadoCalculado));
        setExpressao(String(resultadoCalculado));
      } catch (e) {
        setResultado('Erro');
      }
    } else {
      if (operadores.includes(valor)) {
        if (expressao === '' && valor !== '-') return;

        const ultimoCaractere = expressao.slice(-1);

        if (operadores.includes(ultimoCaractere)) {
          const novaExpressao = expressao.slice(0, -1) + valor;
          setExpressao(novaExpressao);
          setResultado(novaExpressao);
          return;
        }
      }
```

```
      const novaExpressao = expressao + valor;
      setExpressao(novaExpressao);
      setResultado(novaExpressao);
    }
  };

  const Botao: React.FC<BotaoProps> = ({ titulo, corFundo = '#333333', corTexto = '#ffffff' }) => (
    <TouchableOpacity
      style={[styles.botao, { backgroundColor: corFundo }]}
      onPress={() => lidarComToque(titulo)}
    >
      <Text style={[styles.textoBotao, { color: corTexto }]}>{titulo}</Text>
    </TouchableOpacity>
  );

  return (
    <SafeAreaView style={styles.container}>
      <View style={styles.displayContainer}>
        <Text style={styles.textoDisplay} numberOfLines={1} adjustsFontSizeToFit>
          {resultado}
        </Text>
      </View>

      <View style={styles.tecladoContainer}>
        {linhasDeBotoes.map((linha, indexLinha) => (
          <View key={indexLinha} style={styles.linha}>
            {linha.map((botao) => (
              <Botao
                key={botao}
                titulo={botao}
                corFundo={obterCorFundo(botao)}
              />
            ))}
          </View>
        ))}
      </View>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#000000',
  },
  displayContainer: {
    flex: 1,
    justifyContent: 'flex-end',
    alignItems: 'flex-end',
    padding: 20,
  },
  textoDisplay: {
    fontSize: 70,
    color: '#ffffff',
    fontWeight: '300',
  },
  tecladoContainer: {
    paddingBottom: 30,
    paddingHorizontal: 10,
  },
  linha: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 10,
  },
  botao: {
    width: 80,
    height: 80,
    borderRadius: 40,
    justifyContent: 'center',
    alignItems: 'center',
  },
  textoBotao: {
    fontSize: 32,
    fontWeight: '400',
  },
});
```

Execute o projeto com o comando *npm start*, caso esteja na mesma rede utilize o app **Expo Go** para poder testar no seu próprio aparelho.

Personalize o front da aplicação e envie para o **Github** em uma pasta com o nome **ExercicioCalculadoraExpo** no seu repositório.